Lec-12

25/05/26 7:18 PM

Time

Time & Space Complexity

asymptotic notation vs Experimental Way

System

```
public static void main(String[] args) {
		// TODO Auto-generated method stub
		long start=System.currentTimeMillis();
		for (int i = 0; i < 1000_000; i++) {

		}
		long end=System.currentTimeMillis();
		System.out.println(end-start);
	}
```

asymptotic notation → System - Indepent X

[worst] [Avg] [best]

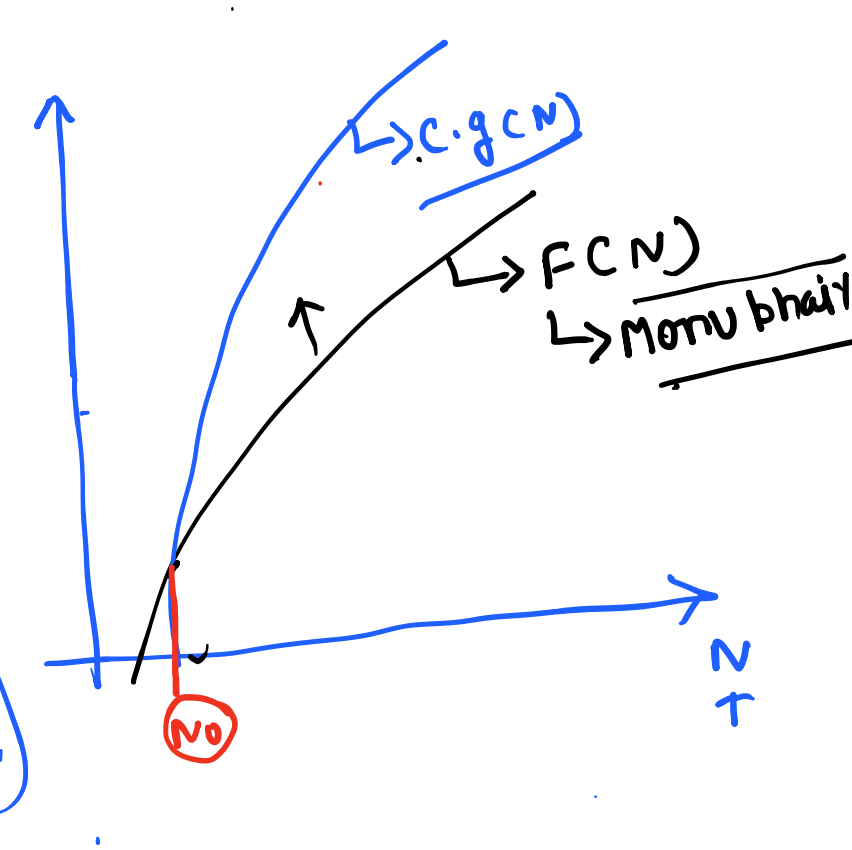
$F(N) \leq c \cdot g(N)$

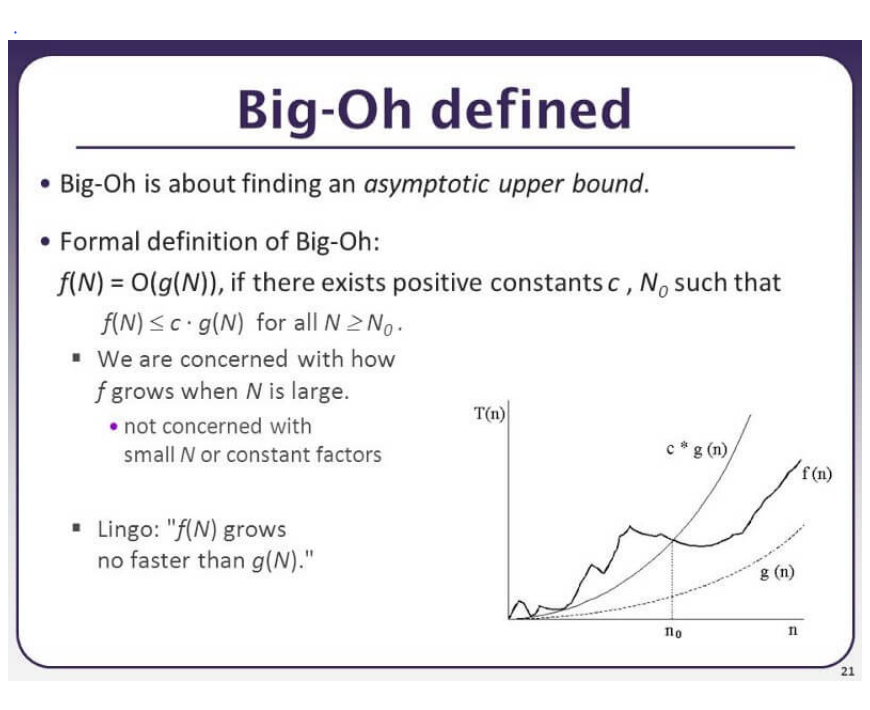
$N \geq N_0$

O(g(N))

Big-O

Big-oh

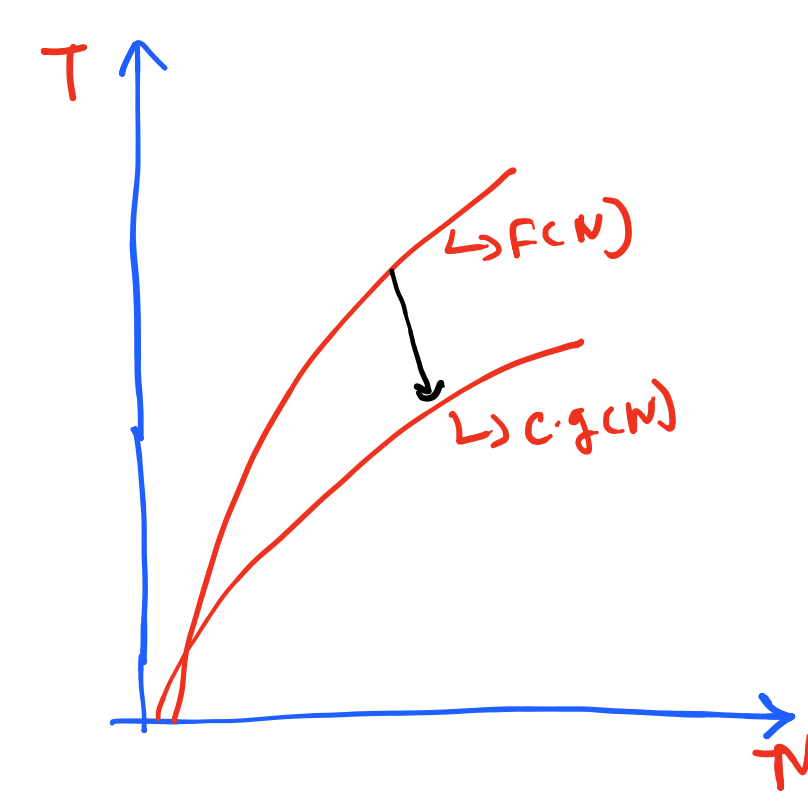




Big-Oh defined

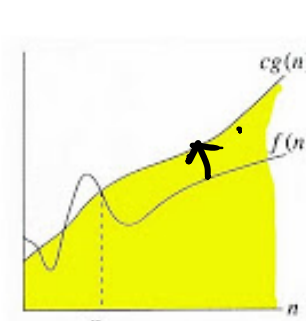
- Big-Oh is about finding an *asymptotic upper bound*.
- Formal definition of Big-Oh:
  f(N) = O(g(N)), if there exists positive constants c , N_0 such that
  $f(N) \leq c \cdot g(N)$ for all $N \geq N_0$.
  - We are concerned with how f grows when N is large.
    - not concerned with small N or constant factors
  - Lingo: "f(N) grows no faster than g(N)."

T(n)
c * g (n)
f (n)
g (n)
N0
n
21

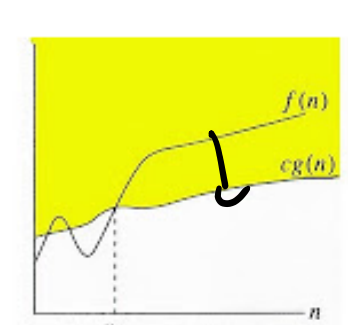


$F(N) >= C \cdot g(N)$

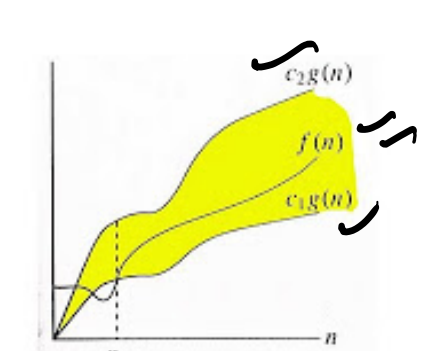
Ω g(n)



Big Oh



Omega



Theta

n^2

$e^K = N$

$e^K = N$

$K = \log_e N$

$T(N) = 3N^2 + 7N + 5$

$T(n) = 3N^2 + 2^N + 7 \cdot 2^N \log N + 7N^3$

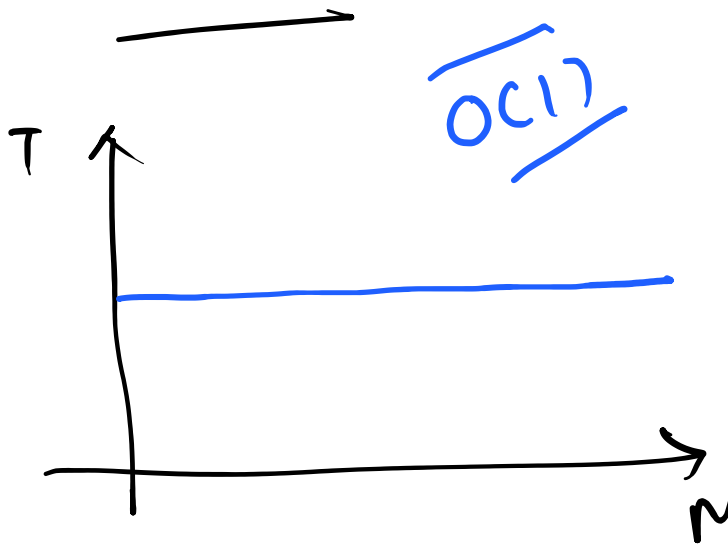
$\frac{N^2}{2} + 3N + 7$

$(2^N \times \log N)$

Code

Constant

O(1)



```
public static int Search(int[] arr, int item) {
		for (int i = 0; i < arr.length; i++) {
			if (arr[i] == item) {
				return i;
			}
		}
		return -1;
	}
```

T(n) = n+1, n+2, n-2

T(N) = N → O(N)

i=0 i=1 i=2 ------ i=n-1

[1 1 1 ... 1]

```
public static int Search(int[] arr, int item) {
		int lo = 0;
		int hi = arr.length - 1;
		while (lo <= hi) {
			int mid = (lo + hi) / 2;
			if (arr[mid] == item) {
				return mid;
			} else if (arr[mid] > item) {
				hi = mid - 1;
			} else {
				lo = mid + 1;
			}
		}
		return -1;
	}
```

O(1)

$2^K = N$

$\frac{N}{2^K} = 1$

$2^K = N$

$K = \log_2 N$

N → 1

$\frac{N}{2}$ → 1

$\frac{N}{2^2}$ → 1

$\frac{N}{2^3}$ → 1

$\frac{N}{2^4}$ → 1

$\frac{N}{2^5}$ → 1

$1 = \frac{N}{2^K}$ → 1

1+1+1 K time

log base

K ? {(log N)} base 2

%

i=1

```
while (i <= n) {
			System.out.println("Hey");
			i *= 2;
		}
```

while (n>0){ n=n/2 }

$2^K = N$ → (log N)

$i=1 \quad i=2 \quad i=2^2 \quad i=2^3 \; ...... \; i=2^K$

1 1 1 1 1

K time

```
while (i <= n) {
			System.out.println("Hey");

			i += 2;
			i += 3;
		}
```

.. (25) (N/5) → O(N)

i=0 i=5 i=10 i=15 i=20 i=25 --- i=100

1 1 1 1 1 1 1

```
while (i <= n) {
			System.out.println("Hey");
			// O(log(N)) base 6
			i *= 2;
			i *= 3;
		}
```

$6^K = N$

$(K = \log_6 N)$

$i=1 \quad i=6 \quad i=6^2 \quad i=6^3 \quad i=6^4 \; --- \; i=6^K$

1 1 1 1 1 1

(K)

```
for (i = 1; i <= n; i++) {
			for (int j = 1; j <= n; j++) {
				System.out.println("hey");
			}
		}
```

→ n

n =

$n \times n \times 1 = n^2$

dependent

i=1 i=2 i=3 i=4 ------ i=N

N N N N N

$n \times n = n^2$

```
for (i = 1; i * i <= n; i++) {
			System.out.println("hey");
		}
```

$i^2 \leq N$

$(i \leq \sqrt{N})$

$O(\sqrt{N})$

③

```
for (i = 1; i <= n; i++) {
			for (int j = 1; j <= i * i; j++) {
				for ( k = 1; k <= n / 2; k++) {
					System.out.println("hey");
				}
			}
		}
```

i=1 i=2 i=3 i=4 ----- i=N

$\left[\frac{N}{2} \quad 2^2 \times \frac{N}{2} \quad 3^2 \times \frac{N}{2} \quad 4^2 \frac{N}{2} \quad N^2 \times \frac{N}{2}\right]$

$\frac{N}{2}\left[1 + 2^2 + 3^2 + 4^2 --- N^2\right]$

$\frac{N}{2} * \left[\frac{N \times (N+1)(2N+1)}{6}\right] = N^4$

```
for (i = n / 2; i <= n; i++) {
			for (int j = 1; j <= n / 2; j++) {
				for ( k = 1; k <= n; k = k * 2) {
					System.out.println("hey");
				}
			}
		}
```

$\frac{n}{2} \cdot \times \frac{N}{2} \cdot \times \log(N)$

$N^2 \log N$

```
for (i = 1; i <= n; i++) {
			for (int j = 1; j <= n; j += i) {
				System.out.println("hey");
			}
		}
```

$2 \to \frac{1}{2}$

i=1 i=2 i=3 i=4 i=5 ...... i=N

$N \quad \frac{N}{2} \quad \frac{N}{3} \quad \frac{N}{4} \quad \frac{N}{5} \quad \frac{N}{N}$

$N\left[1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + ---- \frac{1}{N}\right]$

N log(N)

Sin(x)

Tan(x)

e^n

Sec(x)

$\log(x) = \left[1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} --- \frac{1}{N}\right]$

```
public static void Reverse(int[] arr, int i, int j) {
		// TODO Auto-generated method stub
		while (i < j) {
			int temp = arr[i];
			arr[i] = arr[j];
			arr[j] = temp;
			i++;
			j--;
		}
	}
```

$\frac{N}{2}$

i → ← j

```
public static void Perfix_Sum(int[] arr) {
		// Perfix Array Including Index
		int n = arr.length;
		int[] left = new int[n];
		left[0] = arr[0];
		for (int i = 1; i < n; i++) {
			left[i] = left[i - 1] + arr[i];
		}
		// Sufix Array Including Index
		int[] right = new int[n];
		right[n - 1] = arr[n - 1];
		for (int i = n - 2; i >= 0; i--) {
			right[i] = right[i + 1] + arr[i];
		}
	}
```

O(1)

n

n

TC

n+n=2N → O(n)

S.C → input

2n → O(n)

mini swaping ← bubble sort, Selection sort, Insertion sort → O(n²)

O(1)

O(n)

N^2 ← Constraint

N^3